

Assignment-2

NAME: SHRUTI KHAJURIA

PRN: 202501100011

DIV: SE1

DATA SET: HOUSE PRICE

Dataset Setup Code:

```
House Price.py > house_df
1  import numpy as np
2  import pandas as pd
3
4  np.random.seed(42)
5  n = 100
6
7  house_df = pd.DataFrame({
8      'Id': range(1, n+1),
9      'Area_sqft': np.random.randint(800, 5000, n),
10     'Bedrooms': np.random.randint(1, 6, n),
11     'Bathrooms': np.random.randint(1, 4, n),
12     'Floors': np.random.randint(1, 4, n),
13     'YearBuilt': np.random.randint(1970, 2023, n),
14     'Garage': np.random.choice([0, 1, 2], n),
15     'Location': np.random.choice(['Urban', 'Suburban', 'Rural'], n),
16     'Price': np.random.randint(200000, 2000000, n)
17 })
```

Problem Statements:

Problem 1: Display first 5 rows:

Statement: Load the House Price dataset and display the first 5 records to understand its structure **Code:**

```
print(house_df.head())
```

Output:

```
PS C:\Users\Victus\OneDrive\Desktop\567> & C:\Users\Victus\AppData\Local\Programs\Python\Python314\python.exe c:/Users/Victus/OneDrive/Desktop/567/2006.py
  Id  Area_sqft  Bedrooms  Bathrooms  Floors  YearBuilt  Garage  Location  Price
0    1     1660         5         2        3     2022        2     Rural  1873361
1    2     4572         4         3        2     1970        0     Rural   312816
2    3     3892         5         3        2     1985        0     Urban  1377354
3    4     1266         5         2        3     2008        0  Suburban  1835156
4    5     4244         3         2        2     1974        0     Rural   485977
PS C:\Users\Victus\OneDrive\Desktop\567> █
```

Problem 2: Find shape of the dataset:

Statement: Determine the total number of rows and columns in the House Price dataset.

Code:

```
print(house_df.shape)
```

Output:

```
(100, 9)
```

Problem 3: Statistical summary of Price column:

Statement: Generate descriptive statistics of the 'Price' column including mean, std, min, max and quartiles.

Code:

```
print(house_df['Price'].describe())
```

Output:

```
count    1.000000e+02
mean     1.050764e+06
std      5.050006e+05
min      2.004040e+05
25%      6.557985e+05
50%      1.010362e+06
75%      1.503258e+06
max      1.977489e+06
Name: Price, dtype: float64
```

Problem 4: Check for missing values:

Statement: Identify if any column in the House Price dataset contains null or missing values.

Code:

```
print(house_df.isnull().sum())
```

Output:

```
Id          0
Area_sqft   0
Bedrooms    0
Bathrooms   0
Floors       0
YearBuilt   0
Garage       0
Location     0
Price        0
dtype: int64
```

Problem 5: Calculate average house price:

Statement: Find the mean (average) price of all houses in the dataset using Pandas.

Code:

```
print(round(house_df['Price'].mean(), 2))
```

Output:

```
1050763.52
```

Problem 6: Average price grouped by Location:

Statement: Group records by 'Location' (Urban, Suburban, Rural) and compute the mean house price for each.

Code:

```
print(house_df.groupby('Location')['Price'].mean().round(2))
```

Output:

```
Location
Rural      1056358.82
Suburban   1150932.19
Urban       971205.05
Name: Price, dtype: float64
```

Problem 7: Correlation between Area_sqft and Price:

Statement: Calculate the Pearson correlation coefficient between 'Area_sqft' and 'Price'.

Code:

```
corr_val = house_df['Area_sqft'].corr(house_df['Price'])
print(f"Correlation between Area and Price: {corr_val:.4f}")
```

Output:

```
Correlation between Area and Price: -0.0446
```

Problem 8: Top 5 most expensive houses:

Statement: Retrieve the top 5 houses with the highest prices, showing Id, Area, Location, and Price.

Code:

```
print(house_df.nlargest(5, 'Price')[['Id', 'Area_sqft', 'Location', 'Price']])
```

Output:

	Id	Area_sqft	Location	Price
80	81	3078	Suburban	1977489
6	7	3719	Suburban	1966616
30	31	4356	Suburban	1957266
19	20	2847	Urban	1922320
22	23	2606	Rural	1913796

Problem 9: Add new column 'Age' of house:

Statement: Create a derived column 'Age' representing how old each house is using (2024 - YearBuilt).

Code:

```
house_df['Age'] = 2024 - house_df['YearBuilt']  
print(house_df[['Id', 'YearBuilt', 'Age']].head())
```

Output:

	Id	YearBuilt	Age
0	1	2022	2
1	2	1970	54
2	3	1985	39
3	4	2008	16
4	5	1974	50

Problem 10: Median price by number of Bedrooms:

Statement: Group records by 'Bedrooms' and calculate the median house price for each bedroom count.

Code:


```
print(house_df.groupby('Bedrooms')['Price'].median().round(2))
```

Output:

```
Bedrooms
1      829207.5
2     1110886.0
3      963549.5
4     1000921.0
5     1244375.0
Name: Price, dtype: float64
```

Problem 11: Min and Max Area_sqft using NumPy:

Statement: Find the minimum and maximum area of houses using NumPy's min() and max() functions.

Code:

```
print("Min Area:", np.min(house_df['Area_sqft'].values))
print("Max Area:", np.max(house_df['Area_sqft'].values))
```

Output:

```
Min Area: 834
Max Area: 4917
```

Problem 12: Count houses in each Location:

Statement: Count and display the number of houses available in each location category.

Code:

```
print(house_df['Location'].value_counts())
```

Output:

```
Location
Urban      41
Suburban   31
Rural      28
Name: count, dtype: int64
```

Problem 13: Filter houses where Price > 1,500,000:

Statement: Display all house records where the price exceeds 15 lakh (1,500,000).

Code:

```
filtered = house_df[house_df['Price'] > 1500000]
print(filtered[['Id', 'Location', 'Bedrooms', 'Price']])
```

Output:

	Id	Location	Bedrooms	Price
0	1	Rural	5	1873361
3	4	Suburban	5	1835156
6	7	Suburban	4	1966616
9	10	Rural	3	1583194
18	19	Rural	4	1523483
19	20	Urban	4	1922320
22	23	Rural	2	1913796
30	31	Suburban	4	1957266
32	33	Suburban	1	1755540
33	34	Suburban	3	1698189
35	36	Urban	1	1564652
40	41	Urban	4	1548464
51	52	Rural	1	1684917
52	53	Rural	5	1560642
58	59	Rural	2	1879748
61	62	Suburban	1	1548346
71	72	Suburban	1	1734026
73	74	Urban	4	1570214
76	77	Urban	1	1851451
80	81	Suburban	5	1977489
84	85	Suburban	3	1703678
85	86	Rural	4	1586465
88	89	Urban	3	1564133
90	91	Suburban	4	1553038
96	97	Rural	3	1815952

Problem 14: Most common number of Bedrooms:

Statement: Identify the most frequently occurring number of bedrooms using Pandas mode().

Code:

```
print("Mode of Bedrooms:", house_df['Bedrooms'].mode()[0])
```

Output:

Mode of Bedrooms: 1

Problem 15: Sort by Price descending, show top 10:

Statement: Sort the entire dataset by Price in descending order and display the top 10 records.

Code:

```
print(house_df.sort_values('Price', ascending=False)[['Id', 'Location', 'Price']].head(10))
```

Output:

	Id	Location	Price
80	81	Suburban	1977489
6	7	Suburban	1966616
30	31	Suburban	1957266
19	20	Urban	1922320
22	23	Rural	1913796
58	59	Rural	1879748
0	1	Rural	1873361
76	77	Urban	1851451
3	4	Suburban	1835156
96	97	Rural	1815952

Problem 16: Average Area_sqft grouped by Floors:

Statement: Group records by number of 'Floors' and compute the average area for each group. **Code:**

```
print(house_df.groupby('Floors')['Area_sqft'].mean().round(2))
```

Output:

```
Floors
1      2667.85
2      3092.71
3      2541.81
Name: Area_sqft, dtype: float64
```

Problem 17: Standard deviation of Price using NumPy:

Statement: Compute the standard deviation of the 'Price' column using NumPy's std() function.

Code:

```
std_price = np.std(house_df['Price'].values)
print(f"Std Dev of Price: {std_price:.2f}")
```

Output:

```
Std Dev of Price: 502469.27
```

Problem 18: Filter houses with Garage=2 and Bedrooms>=4:

Statement: Apply multiple conditions to filter houses with exactly 2 garages and at least 4 bedrooms.

Code:

```
result = house_df[(house_df['Garage'] == 2) & (house_df['Bedrooms'] >= 4)]
print(result[['Id', 'Bedrooms', 'Garage', 'Location', 'Price']])
```

Output:

	Id	Bedrooms	Garage	Location	Price
0	1	5	2	Rural	1873361
15	16	5	2	Suburban	200404
19	20	4	2	Urban	1922320
23	24	4	2	Urban	435365
47	48	4	2	Rural	1000921
52	53	5	2	Rural	1560642
59	60	4	2	Urban	325917
65	66	5	2	Suburban	481342
68	69	4	2	Urban	201252
83	84	4	2	Suburban	980722
90	91	4	2	Suburban	1553038

Problem 19: Replace Location values:

Statement: Rename location values: Urban -> City, Suburban -> Town, Rural -> Village using replace().

Code:

```
house_df['Location_New'] = house_df['Location'].replace({
    'Urban': 'City', 'Suburban': 'Town', 'Rural': 'Village'
})
print(house_df[['Location', 'Location_New']].head())
```

Output:

	Location	Location_New
0	Rural	Village
1	Rural	Village
2	Urban	City
3	Suburban	Town
4	Rural	Village

Problem 20: Percentage of houses in each Location:

Statement: Calculate the percentage distribution of houses across all location types.

Code:

```
pct = house_df['Location'].value_counts(normalize=True).mul(100).round(2)
print(pct)
```

Output:

```
Location
Urban      41.0
Suburban   31.0
Rural      28.0
Name: proportion, dtype: float64
```